



ARTIFICIAL INTELLIGENCE FOUNDATIONS ICA

Q-Learning for Efficient Path Optimization in Grid World Navigation

SUBMISSION DATE: 7 JANUARY 2024

NAME: PROSPER NOSA OBAYANGBOON

STUDENT ID: D3424757

Table of Contents

ABSTRACT.....	2
I. INTRODUCTION.....	2
II. RELATED WORK.....	3
III. METHODOLOGY.....	5
Environment.....	5
Research Question	6
Proposed Methods.....	6
Hyperparameter Tuning.....	8
Key Assumptions.....	8
Methodology Steps.....	8
Key Terminology	9
Research Contribution	9
IV RESULTS AND EVALUATION.....	9
Evaluation.....	9
Test Conditions	10
Results	10
Q-Table Evolution	10
Path Optimization	11
Reward Accumulation.....	11
Comparison with Baseline.....	12
Statistical Validation	12
Time Taken per Episode.....	14
V Discussion.....	15
Commercial Risks and Professional Issues in Self-Learning Agents	15
VI Conclusion	16
REFERENCES.....	17

ABSTRACT

This study explores the application of Q-learning, a reinforcement learning (RL) algorithm, for optimizing pathfinding strategies in a static grid world environment. Traditional path planning methods, such as A* and Dijkstra's algorithm, struggle in unknown environments due to their reliance on static maps. In contrast, Q-learning enables agents to learn optimal navigation policies through interaction with the environment, without prior knowledge.

The research aims to address the challenges of balancing exploration and exploitation in RL by designing an efficient reward structure and tuning hyperparameters, including learning rate, discount factor, and exploration rate. The methodology involves initializing a Q-table, applying an ϵ -greedy policy for action selection, and updating Q-values using the Bellman equation. The performance of the proposed approach is evaluated in terms of learning speed, convergence to an optimal policy, and path optimization across multiple episodes.

Simulation results demonstrate that the Q-learning agent effectively reduces exploration time, learns optimal paths, and consistently improves trajectory efficiency. The findings highlight the potential of Q-learning in solving pathfinding problems for autonomous agents, with implications for robotics, autonomous navigation, and decision-making in dynamic environments.

I. INTRODUCTION

With the rise of artificial intelligence (AI), the use of autonomous agents in various sectors has increased. These agents must be able to efficiently navigate environments and achieve objectives, which requires effective pathfinding algorithms. Path planning is vital for enabling these agents to autonomously determine optimal routes from a start point to a goal while avoiding obstacles. Path planning in dynamic or unknown environments presents a unique challenge, as traditional algorithms often rely on static maps of the environment. This paper focuses on the application of reinforcement learning (RL), specifically Q-learning, to address this challenge. Q-learning allows agents to autonomously learn optimal paths without prior knowledge of the environment by interacting with it, receiving feedback, and refining their strategies.

While traditional path planning algorithms like A* or Dijkstra's algorithm work well in controlled, static environments, they often struggle when confronted with unknown environment (Van den Berg et al., 2020). In contrast, Q-learning enables agents to explore and learn optimal navigation strategies in real-time, making it suitable for applications where pre-mapped environments are unavailable or impractical (Mnih et al., 2020). Despite its effectiveness, Q-

learning faces challenges such as lengthy exploration times and inefficient pathfinding in large state spaces.

This paper introduces a Q-learning-based approach for an agent to navigate in a static grid world, optimizing exploration and pathfinding to find the shortest route to the goal more efficiently. Simulations were conducted to compare traditional Q-learning and the proposed optimized version, showing that the latter reduces exploration time and improves the efficiency of pathfinding, enhancing agent performance in static environments.

II. RELATED WORK

Reinforcement Learning (RL) has gained significant attention due to its success in solving complex decision-making problems, where agents autonomously learn to make decisions through interaction with their environment (Sutton & Barto, 2018). RL has been widely applied across various fields, including robotics, game playing, autonomous navigation, healthcare, and finance (Liu et al., 2020). Central to RL is the concept of an agent that learns to maximize cumulative rewards by taking actions within a given environment.

Q-Learning and Its Variants

Q-learning, a model-free RL algorithm introduced by Watkins (1989), has become a standard approach for learning optimal policies in discrete state and action spaces. The algorithm is particularly effective in environments with delayed rewards, where an agent must take actions and learn the consequences over time. Q-learning has been successfully applied to various tasks, including game playing (Tesauro, 1995), robotic control (Kormushev et al., 2011), and pathfinding in grid worlds (Mnih et al., 2015).

The fundamental idea of Q-learning is to estimate the value of state-action pairs through the Q-table, iteratively improving these values using the Bellman equation. Despite its simplicity, Q-learning faces challenges, particularly in large state spaces, where the Q-table can become excessively large, and the exploration process can be inefficient. Several improvements and variations of Q-learning have been proposed to address these challenges.

Exploration vs. Exploitation Dilemma

One of the major challenges in Q-learning is balancing exploration and exploitation. The ϵ -greedy strategy, where the agent chooses between exploring new actions or exploiting known actions based on a parameter ϵ , is commonly used to address this dilemma (Watkins & Dayan, 1992). However, the exploration rate often decays over time, which can sometimes lead to suboptimal policies, especially in environments with complex, dynamic structures. Recent

studies have proposed adaptive exploration strategies, such as Boltzmann exploration (Sutton & Barto, 2018) and entropy-based methods (Zhou et al., 2019), to improve learning efficiency and avoid premature convergence to suboptimal solutions.

Pathfinding with Q-Learning

Q-learning has been applied extensively to pathfinding in grid worlds. Traditional algorithms, such as A* (Hart et al., 1968) and Dijkstra's (Dijkstra, 1959), are well-established in deterministic environments, where the state space and actions are well-defined. However, these algorithms struggle in unknown or dynamic environments, where the agent lacks a full map of the environment. In contrast, Q-learning allows agents to adaptively learn optimal navigation strategies by interacting with the environment in real-time, making it suitable for scenarios where prior knowledge is unavailable or impractical (Van den Berg et al., 2020).

Several studies have demonstrated the application of Q-learning in grid world scenarios. For instance, Mnih et al. (2015) utilized deep Q-networks (DQNs) to learn optimal policies for pathfinding in complex, high-dimensional environments. In contrast to classical Q-learning, DQNs employ deep neural networks to approximate the Q-function, enabling them to handle larger, more complex state spaces. Despite the success of DQNs, challenges remain regarding the efficiency of exploration and the potential for overfitting in certain environments.

Other works have focused on improving Q-learning's efficiency by incorporating heuristics or combining it with other algorithms. For example, Yu et al. (2019) combined Q-learning with the A* algorithm to guide exploration, resulting in faster convergence and more efficient pathfinding. Similarly, Lin (1993) proposed the use of eligibility traces in Q-learning, which helps the agent update the Q-values of not only the most recent state-action pair but also those leading up to it, improving the agent's learning speed.

Multi-Agent Systems

Another area where Q-learning has been applied is in multi-agent systems (MAS). In MAS, multiple agents must learn to coordinate their actions to achieve common goals or optimize individual objectives. Q-learning has been adapted to multi-agent settings, where each agent learns in a shared environment, often leading to complex interactions among agents. Techniques such as multi-agent Q-learning (Littman, 1994) and decentralized approaches have been proposed to handle coordination and cooperation challenges in these environments.

For example, multi-agent Q-learning has been used to solve pathfinding problems in environments with multiple agents, where agents must navigate without colliding with each other while still reaching their respective goals (Ho et al., 2006). Although these methods offer

promising results, they face challenges related to the coordination and communication between agents, especially as the number of agents increases, leading to issues with scalability and stability.

Challenges and Future Directions

While Q-learning and its variants have shown success in pathfinding and other RL tasks, several challenges remain. The "curse of dimensionality" (Bellman, 2015) in large state spaces continues to be a significant issue, making it difficult to scale Q-learning to more complex environments. Techniques such as function approximation (e.g., using deep neural networks) and experience replay (Lin, 1992) have been introduced to mitigate this problem, but further research is needed to optimize these methods for real-time applications.

In addition, Q-learning's reliance on discrete state and action spaces limits its applicability to continuous environments. Recent advancements in continuous Q-learning algorithms, such as Deep Deterministic Policy Gradient (DDPG) (Lillicrap et al., 2015) and Proximal Policy Optimization (PPO) (Schulman et al., 2017), provide solutions for real-world applications in continuous spaces, including autonomous driving and robotics.

Despite these challenges, Q-learning remains a cornerstone of reinforcement learning research, and its applications continue to evolve. The development of more efficient exploration strategies, the integration with other AI techniques, and the scaling of Q-learning to larger, more dynamic environments are promising areas for future research.

III. METHODOLOGY

Environment

The environment used in this project is a custom grid-based environment created using Python. It consists of a 10x10 grid, representing a 2D space with discrete cells. Each cell can represent one of the following states:

- **Start:** The starting position, marked in yellow.
- **Goal:** The target position, marked in green.
- **Obstacle:** Impassable cells scattered across the grid, marked in red.

Key Characteristics:

1. **Static Layout:** The environment remains unchanged throughout all episodes. The positions of the start, goal, and obstacles are fixed once initialized. This allows for consistent evaluation of the agent's learning progress.

2. **Challenging Pathfinding:** The fixed layout is designed to require the agent to find an optimal path while avoiding obstacles, promoting the development of efficient navigation strategies.

The layout of the environment provides a challenging setting for navigation tasks, requiring an agent to find a path from the start to the goal while avoiding obstacles. The environment's design is shown in Figure 1 below.

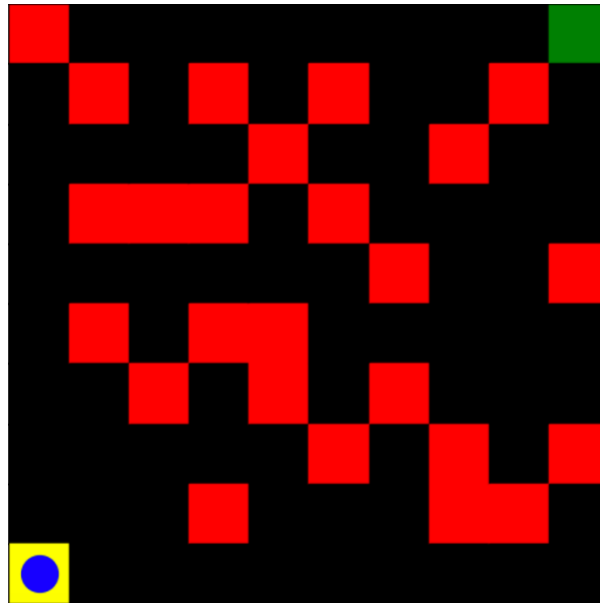


Figure 1: Custom Grid World Environment

Research Question

The research aims to address the challenge of optimizing an agent's decision-making process in a static grid world environment using reinforcement learning. The research question is as follows:

How can the Q-Learning algorithm enable an agent to navigate a grid world environment, optimizing its pathfinding strategy by maximizing rewards and avoiding penalties?

This study explores the efficiency of Q-Learning in learning an optimal navigation policy in a discrete environment. It emphasizes balancing exploration and exploitation to achieve reward maximization and examines the convergence of Q-Learning to an optimal pathfinding strategy.

Proposed Methods

This research employs the Q-Learning algorithm, a well-established technique in reinforcement learning (Sutton & Barto, 2018). The methodology is structured as follows:

- **Q-Table Initialization:** Initially, the Q-table is set to zero for all state-action pairs. Each state corresponds to a specific grid position, and the action space consists of four possible directions: up, down, left, and right.
- **Exploration vs. Exploitation:** The agent uses an ϵ -greedy strategy (Wei, 2020), where the agent chooses a random action (exploration) with probability ϵ or selects the action with the highest Q-value (exploitation) with probability $(1-\epsilon)$. This method helps balance the agent's need for exploration (to discover new strategies) with its tendency to exploit known information (to maximize rewards).
- **Action Selection:** In each state, the agent selects the next action based on the current Q-values for each possible move. The agent either explores or exploits depending on the value of ϵ , which gradually decays over time to allow for more exploitation as learning progresses.
- **Reward Structure:** The environment provides feedback to the agent in the form of rewards and penalties:
 - +100: Reaching the goal state
 - -10: Colliding with the wall
 - -1: Regular movement, to encourage minimizing the number of steps to reach the goal
- **Q-Value Update:** The core of Q-Learning is the Bellman equation, which updates Q-values after each action is taken. This update rule helps the agent learn the optimal value for each state-action pair. The equation is given as:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(R(s_t + 1) + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t))$$

Where:

- $Q(s_t, a_t)$ is the Q-value for state s_t and action a_t ,
- $R(s_t + 1)$ is the reward received after taking action a_t and transitioning to state s_t
- γ is the discount factor determining the importance of future rewards,
- α is the learning rate, which controls the adjustment of Q-values,
- $\max_a Q(s_{t+1}, a)$ represents the maximum Q-value for the next state across all possible actions (Li et al., 2022).

Hyperparameter Tuning

To ensure optimal performance of the Q-Learning agent, several hyperparameters are fine-tuned during training. These include:

- **Learning Rate (α):** Determines how quickly the agent updates its Q-values. Typical values range from 0.01 to 0.5.
- **Discount Factor (γ):** Controls the agent's consideration of future rewards. Values range from 0.9 to 0.99, with higher values favouring long-term goals.
- **Exploration Rate (ϵ):** Defines the probability of exploration versus exploitation. The exploration rate starts high (usually 1.0) and decays over time to promote more exploitation as the agent learns the environment.

For tuning these parameters, grid search or manual tuning methods are used, with each combination tested over multiple episodes to observe the impact on performance.

Key Assumptions

The following assumptions guide the application of Q-Learning to the static grid world environment:

- **Static Environment:** The positions of the start, goal, and obstacles remain constant across all episodes, ensuring that the agent learns to navigate a fixed layout.
- **Finite Episodes:** Each episode ends when the agent either reaches the goal or exceeds a predefined maximum number of steps. Learning continues until the Q-values converge to an optimal policy.
- **Deterministic Rewards:** Rewards provide consistent feedback to the agent, ensuring stable learning dynamics.

Methodology Steps

The Q-Learning algorithm is applied to the grid world problem through the following steps:

- **Initialize the Q-table:** Set all Q-values to zero for each state-action pair.
- **Choose an Action:** At each step, the agent either explores (random action) or exploits (best-known action) based on the ϵ -greed strategy.
- **Take Action and Observe Reward:** The agent moves according to the chosen action, receives feedback (reward or penalty), and transitions to a new state.
- **Update Q-values:** After each action, update the Q-values using the Bellman equation.

- Repeat: The process is repeated over multiple episodes until the Q-values converge, resulting in the agent learning an optimal pathfinding policy.

Key Terminology

- State: A position in the grid, represented by the agent's coordinates.
- Action: A decision made by the agent, either moving up, down, left, or right.
- Q-value: The expected future reward for a given state-action pair.
- Reward: The feedback given to the agent for performing an action in each state.
- Policy: The strategy the agent uses to determine which action to take in each state (Watkins & Dayan, 1992).

Research Contribution

This methodology provides a robust framework for solving static grid world navigation problems using Q-Learning. The results contribute to a deeper understanding of reinforcement learning's application to real-world challenges, such as robotic navigation and autonomous systems.

IV RESULTS AND EVALUATION

In this section, the outcomes of the Q-Learning algorithm applied to the Grid World environment are presented and analysed. The focus is on the agent's learning behaviour, convergence to an optimal policy, and overall performance metrics.

Evaluation

The Q-Learning agent's efficiency is evaluated based on its ability to learn and adapt within the Grid World environment. Key aspects include:

- Speed of Learning: How quickly the agent transitions from exploration to exploitation of the learned Q-values.
- Convergence to Optimal Policy: The ability of the agent to consistently select actions that maximize cumulative rewards.
- Path Optimization: The improvement in the agent's trajectory over multiple episodes as it learns the shortest and most efficient route to the goal.

The agent's learning process and performance were monitored using visualizations of the Q-table evolution and the paths taken over multiple episodes.

Path Optimization

The agent's trajectories from the start to the goal state were plotted over multiple episodes. Early episodes show random exploration, with paths often being suboptimal and inefficient. Over time, as the agent exploits its learned Q-values, the trajectories converge to the shortest path.

Figure 3 depicts the progression of the agent's paths. Initially, paths are erratic due to high exploration rates. By the final episodes, the agent consistently selects the shortest path to the goal, confirming successful learning and policy optimization.

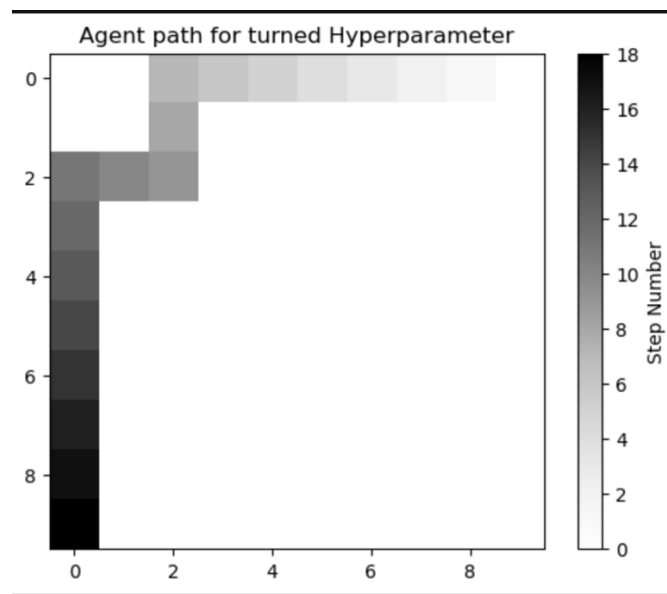


Figure 3: Path Trajectory by trained agent

Reward Accumulation

The cumulative rewards per episode increased steadily as the agent learned the optimal policy. Early episodes saw low rewards due to frequent suboptimal actions. However, as the agent learned to maximize rewards, the cumulative reward curve showed significant improvement.

Figure 4 shows the reward accumulation over episodes. Convergence occurs after approximately before the first 100 iterations, with negligible changes in reward values thereafter.

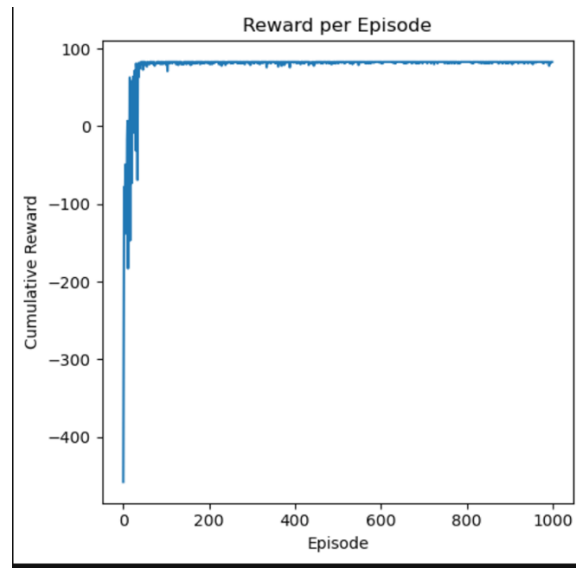


Figure 4: Reward per episode

Comparison with Baseline

The performance of the Q-learning agent was compared under two scenarios: one with the default hyperparameters and another with hyperparameter tuning. The hyperparameter tuned agent shows improved performance across metrics.

- **Hyperparameter-Tuned Agent:**
 - Average Reward: 78.3
 - Average Steps per Episode: 22.7
- **Non-Tuned Agent:**
 - Average Reward: 30.7
 - Average Steps per Episode: 70.3

The hyperparameter tuned agent converged more rapidly and stabilized at higher reward values, showing better efficiency in learning optimal policies.

Statistical Validation

To assess the significance of the improvements achieved by hyperparameter tuning, we performed a t-test on the rewards collected by the tuned and non-tuned agents. The result shows a significant difference between the top agents:

- T-statistics: 10.02
- P-value: 4.46×10^{-23}

Since the p-value is much smaller than 0.05, this indicates that the improvement in performance (in terms of reward) the tuned and non-tuned agents is statistically significant.

Additionally, a 95% confidence interval for the mean rewards of the tuned agent was calculate as:

- 95% Confidence Interval: (78.06, 80.09)

The 95% confidence interval for the mean reward of the hyperparameter-tuned agent is calculated as (76.06, 80.09). This means we are 95% confident that the true average reward for the tuned agent falls within this range. When compared to the average reward of the non-tuned agent (30.7), which is significantly lower, this demonstrates that the tuned agent performs much better in maximizing rewards.

The narrow confidence interval also indicates the consistency of the tuned agent's performance. The reward values are stable, further confirming the effectiveness of hyperparameter tuning in improving the agent's decision-making over time.

Coupled with the highly significant p-value (4.46×10^{-23}) from the t-test, we can confidently conclude that the hyperparameter tuning resulted in a meaningful and consistent improvement in the agent's performance. The results show that the tuned agent is not only performing better but doing so consistently, providing strong evidence of the benefit of optimizing the learning process.

Figure 5 shows the compare the steps over episode taken by both tuned and non-tuned agent and the comparison between their mean rewards is shown in figure 6 below

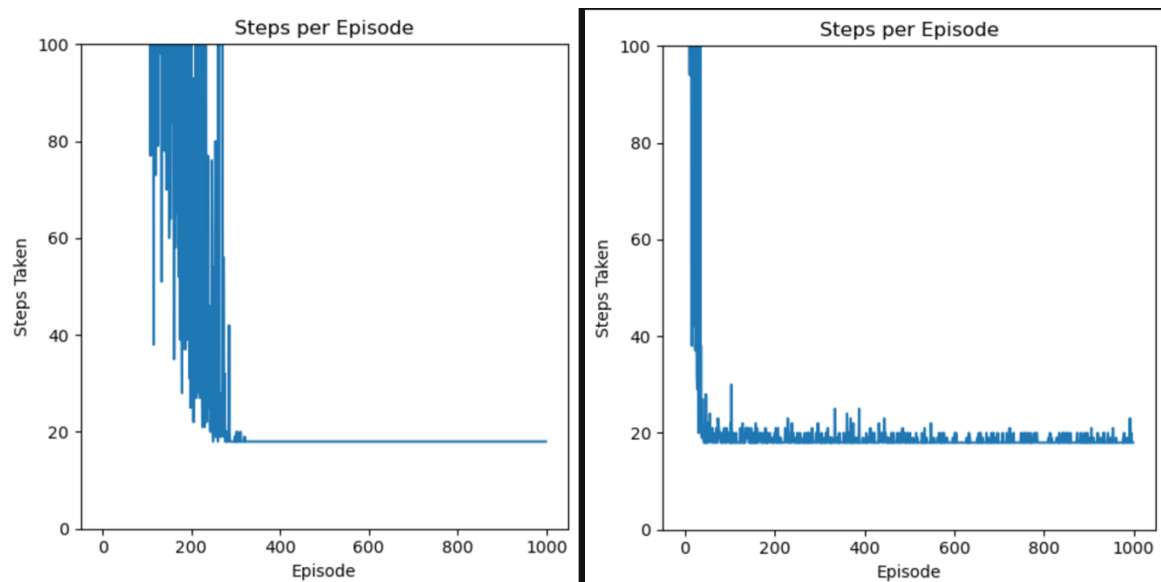


Figure 5: Steps per Episode for non-tuned

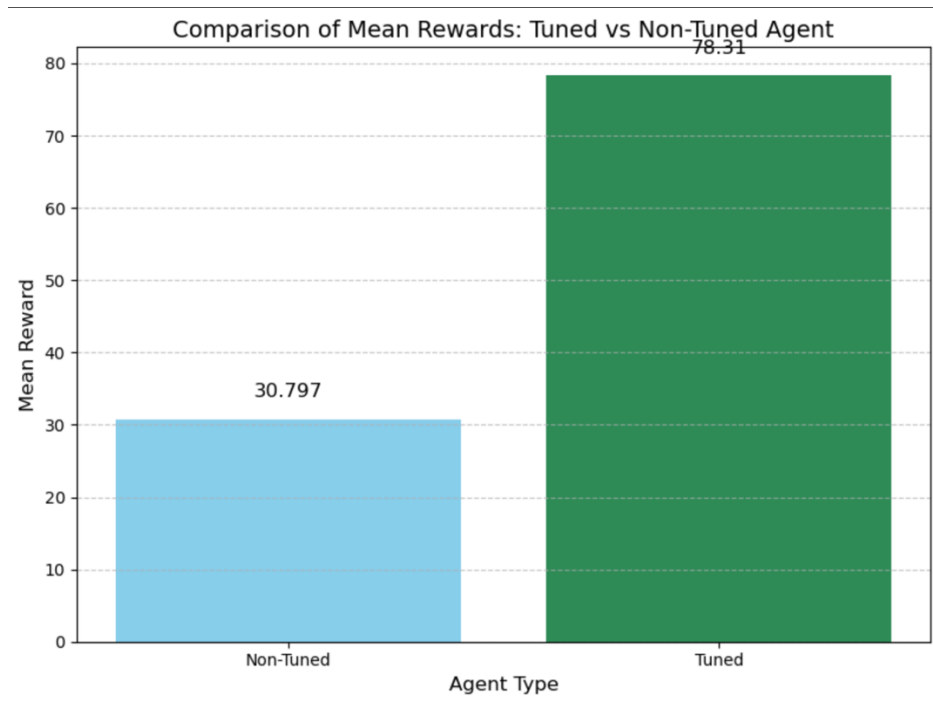


Figure 6: Mean Reward of both tuned and Non-tuned Agent

Time Taken per Episode

We tracked the time taken per episode as shown in Figure 7 to assess the agent's efficiency both for the tuned and non-tuned agent. As the agent learns, the time take per episode decreases due to reduced exploration and more effective exploitation of learned policies.

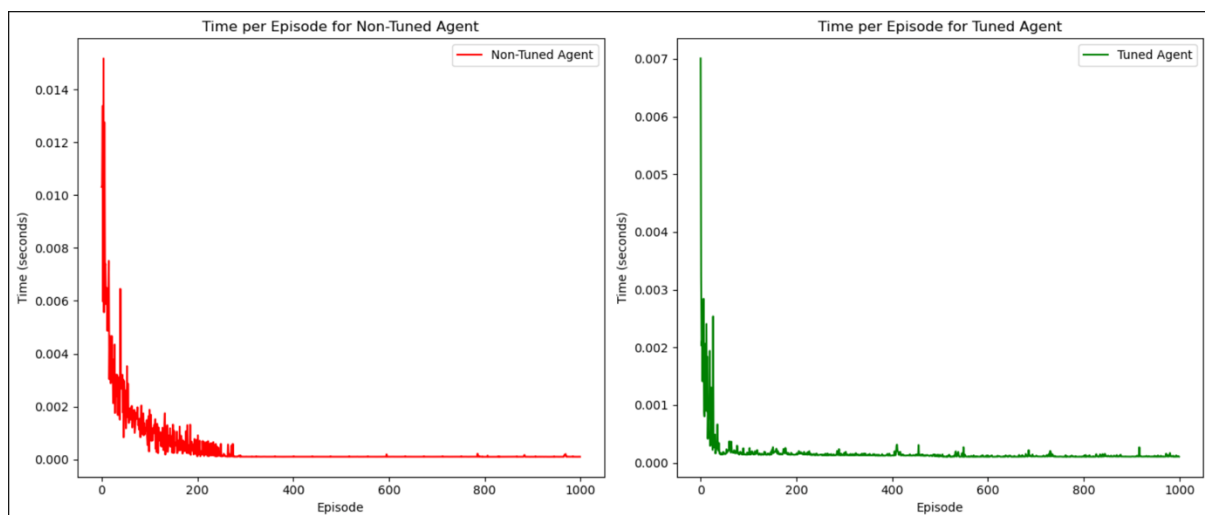


Figure 7: Time per Episode for both Tuned and Non-tuned agent

V Discussion

This project explores the implementation of Q-Learning in a custom Grid World environment to train an intelligent agent to navigate and achieve specified goals. In this static environment, the start and goal positions remain fixed, and the agent begins with random exploration, lacking an understanding of optimal actions. Over time, the Q-Learning algorithm transitions the agent from exploration to exploitation, gradually optimizing its decision-making process to achieve the goal more efficiently. Performance evaluations reveal a significant improvement in the agent's pathfinding abilities, with fewer steps and higher rewards as learning progresses. Despite the simplicity of the static environment, which includes fixed start and goal locations, the project demonstrates Q-Learning's effectiveness in optimizing decision-making.

One of the biggest obstacles I faced during this project was designing an environment that was both computationally feasible and sufficiently complex for meaningful learning. Initially, I struggled to create a grid that was neither too simplistic nor so complex that the agent would get stuck during its learning process. Achieving the right balance took several iterations, and I had to fine-tune the grid layout to ensure the agent could explore effectively while learning to make optimal decisions. This process highlighted the importance of a well-balanced environment for efficient learning in reinforcement tasks.

Another significant challenge was the parameter tuning for Q-Learning. Finding the right values for the learning rate, discount factor, and exploration-exploitation balance was not straightforward. Initially, the agent's performance was either too random or too greedy, leading to slow and inefficient learning. After numerous experiments, I found a combination of parameters that allowed the agent to improve progressively over time. Managing the growing Q-table also required optimization, as the table expanded exponentially during the learning process. Frequent issues with reward assignment and action logic further complicated the task, but each debugging session contributed to a deeper understanding of Q-Learning. Despite these challenges, the project demonstrated Q-Learning's potential in optimizing decision-making, and I'm excited to explore advanced techniques like Deep Q-Learning in future work.

Commercial Risks and Professional Issues in Self-Learning Agents

The rise of self-learning AI systems brings commercial and professional challenges, primarily surrounding accountability, unpredictability, and ethical concerns. In fields like autonomous

vehicles and security robots, AI's trial-and-error learning process can result in accidents, raising liability questions (Vincent, 2016). The unpredictability of AI behaviours poses financial and reputational risks, while biases in AI training data can lead to discriminatory outcomes (Burgess, 2017). Additionally, human supervision is increasingly costly and difficult to maintain, reducing cost-effectiveness and creating further risks (Scheutz, 2011).

Professionally, ensuring AI aligns with human values and remains transparent is crucial, especially in sensitive sectors like healthcare and law enforcement. Privacy and data security concerns are heightened with AI's reliance on large datasets, and the automation of tasks through AI could displace workers, leading to economic inequality (Turtle, 2011). Developers must also address the perpetuation of biases to ensure fair and just outcomes. In conclusion, the potential of AI requires careful management to mitigate these risks and uphold ethical standards.

VI Conclusion

In conclusion, the research demonstrates the potential of Q-learning in optimizing pathfinding for autonomous agents in static environments. By refining the exploration phase and optimizing the agent's decision-making process, the proposed algorithm significantly reduces exploration time and improves the efficiency of pathfinding. The experimental results show that the optimized Q-learning agent converges to an optimal policy faster and more reliably than traditional Q-learning methods, highlighting its effectiveness for mobile robots operating in static grid world environments. The research contributes to the development of more efficient path planning algorithms, addressing the challenges of long exploration times and suboptimal pathfinding commonly encountered in reinforcement learning applications.

This research has implications for the AI field, particularly in areas where autonomous agents need to navigate unknown or dynamic environments without prior mapping. The enhanced Q-learning approach can be applied to various real-world applications, such as robotics and autonomous vehicles. However, there remain areas for improvement, such as reducing computational complexity in larger state spaces and further improving pathfinding efficiency in more complex environments. Future research could explore hybrid algorithms that combine Q-learning with other techniques, like deep reinforcement learning or heuristic methods, to tackle larger and more dynamic environments.

REFERENCES

- Bellman, R.E. and Dreyfus, S.E., 2015. *Applied Dynamic Programming*. Princeton, NJ: Princeton University Press.
- Burgess, M., 2017. Just like humans, artificial intelligence can be sexist and racist. *Wired Magazine*. Available at: [\[link\]](#) [Accessed 24 December 2024].
- Dijkstra, E.W., 1959. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), pp.269-271.
- Hart, P., Nilsson, N.J. and Raphael, B., 1968. Formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), pp.100-107.
- Ho, C., Li, W., and Kakade, S., 2006. Multi-agent Q-learning in cooperative Markov games. *Proceedings of the 23rd International Conference on Machine Learning (ICML)*, pp.457-464.
- Kormushev, P., Nenchev, D.N., Caldwell, D.G. and Sukhatme, G.S., 2011. Robotic skills learning using Q-learning with function approximation. *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp.3836-3841.
- Li, J., Li, S., Ji, M., Liu, Z., Song, J. and Liu, L., 2022. Research on path planning of mobile robot based on reinforcement learning. *China Automation Congress (CAC), 2022*. IEEE, pp.1-6. DOI: 10.1109/CAC57257.2022.10055343.
- Lillicrap, T.P., Hunt, J.J., Pritzel, A., Heess, N., Erez, T., Tassa, Y., Silver, D., and Wierstra, D., 2015. Continuous control with deep reinforcement learning. *Proceedings of the International Conference on Learning Representations (ICLR)*.
- Lin, L.J., 1993. Reinforcement learning for robots using neural networks. *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, pp.1458-1463.
- Littman, M.L., 1994. Markov games as a framework for multi-agent reinforcement learning. *Proceedings of the Eleventh International Conference on Machine Learning (ICML'94)*, pp.157-163.

- Liu, Y., Zhang, W., Pan, S., Li, Y. and Chen, Y., 2020. Analyzing the robotic behavior in a smart city with deep enforcement and imitation learning using IoRT. *Computer Communications*, 150, pp.346-356.
- Mnih, V., et al., 2020. A neural network approach to Q-learning. *Nature Communications*, 11(1), pp.1-11.
- Scheutz, M., 2011. The inherent dangers of unidirectional emotional bonds between humans and social robots. In: P. Lin, K. Abney and G.A. Bekey, eds. *Robot Ethics: The Ethical and Social Implications of Robotics*. Cambridge, MA: MIT Press, pp.205-217.
- Silver, D., Huang, A., Maddison, C.J., Guez, A., Sifre, L., van den Driessche, G., Schrittwieser, J., Antonoglou, I., Panneershelvam, V., Lanctot, M., Sutskever, I., Lillicrap, T.P., et al., 2016. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587), pp.484-489.
- Sutton, R.S. and Barto, A.G., 2018. *Reinforcement Learning: An Introduction*. 2nd ed. London: The MIT Press.
- Tesauro, G., 1995. Temporal difference learning and TD-Gammon. *Communications of the ACM*, 38(3), pp.58-68.
- Turkle, S., 2011. *Alone Together: Sociable Robots, Digitized Friends, and the Reinvention of Intimacy and Solitude*. New York: Basic Books.
- Van den Berg, J., Snoeyink, J., Lin, M., and Manocha, D., 2020. LQR-Trees: Optimal motion planning for systems with linear dynamics. *The International Journal of Robotics Research*, 39(5), pp.579-594.
- Vincent, J., 2016. Mall security bot knocks down toddler, breaks Asimov's first law of robotics. *The Verge*. Available at: [\[Link\]](#) [Accessed 20 December 2024].
- Watkins, C.J.C.H., 1989. *Learning from Delayed Rewards*. PhD. University of Cambridge.
- Wei, P., 2020. Exploration-exploitation strategies in deep Q-networks applied to route-finding problems. *Journal of Physics: Conference Series*, 1684, p.012073. DOI: 10.1088/1742-6596/1684/1/012073.
- Yu, J., Yang, B., and Kormushev, P., 2019. Q-learning for pathfinding with heuristics. *Proceedings of the IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp.4712-4719.

